Cahier des charges — Application de gestion d'événements

Module : Programmation Orientée Objet (Java)
Rendu : Étape 1 (sujet, cahier des charges, description, conception UML), complété au jalon 2 (base de données, section 8)
Formation : 3A FISA Informatique — groupe G1
Membres : Vince BONIFACE, Théo MARTIN, Romain NOTTEAU, Lony MICHAUX

---

1. Choix du sujet

Nous avons choisi de développer une **application client/serveur de gestion d'événements** (anniversaires, mariages, soirées…).

Un organisateur crée un événement, invite des personnes et liste ce qu'il faut acheter ou apporter (boissons, enceinte, éclairage…). Chaque invité répond à son invitation, peut prendre en charge l'achat d'un besoin et échanger des messages avec les autres participants.

Pourquoi ce sujet :

- **Héritage et polymorphisme naturels :** plusieurs types de besoins (`Consommable`, `Materiel`, lui-même décliné en `MaterielSon` et `MaterielLumiere`) partagent des attributs et un comportement communs (`acheter()`, `calculerCout()`) et sont manipulés ensemble dans une `List<Besoin>`, quel que soit leur type.
- **Encapsulation et règles métier :** une invitation change d'état (en attente → acceptée / refusée), un besoin n'est attribué qu'à un participant, seuls les organisateurs invitent ; ces règles sont protégées dans les classes.
- **Réseau :** plusieurs personnes consultent et modifient les mêmes données, ce qui justifie une architecture client/serveur avec Sockets Java.
- **Utile au quotidien :** le besoin est concret (« qui achète quoi, pour quel budget ? ») et facile à démontrer.

---

2. Description du projet

2.1 Contexte

Organiser un événement entre amis ou en famille passe souvent par des groupes de messagerie : les réponses se perdent, on ne sait pas qui vient, et deux personnes achètent la même chose pendant que personne ne pense aux gobelets. Ce projet s'inscrit dans le module de POO en Java : l'objectif

pédagogique est de mettre en œuvre une conception objet solide (héritage, polymorphisme, encapsulation, exceptions) dans une application réseau simple.

2.2 Objectifs

1. Centraliser les informations d'un événement (nom, date, lieu, description, prix, visuels) en un seul endroit.
2. Savoir à tout moment qui est invité, qui a accepté et qui a refusé.
3. Répartir les achats entre participants pour éviter oublis et doublons, et connaître le budget total.
4. Permettre aux participants d'échanger des messages au sein de l'événement.
5. Garantir que chacun n'agit que sur ce qui le concerne (contrôle d'accès).

2.3 Utilisateurs cibles

Rôle	Description
Visiteur	Personne non connectée : peut uniquement se connecter.
Utilisateur connecté	Personne connectée : consulte ses événements, les besoins et ses messages, crée un événement (elle en devient organisatrice).
Organisateur	Personne qui a créé l'événement ou qui a été ajoutée comme co-organisateur : administre, invite, gère les besoins.
Invité	Personne ayant reçu une invitation : y répond et, si elle a accepté, peut prendre en charge un besoin.

Une même personne peut être organisatrice d'un événement et invitée à un autre : le rôle dépend de l'événement, pas du compte. Ce choix correspond à l'usage réel (chacun organise ses propres événements) ; en base, il se traduit par la table de jonction `organisateur` et par la table `invitation` (section 8).

2.4 Attentes

- Une application **fonctionnelle et démontrable** : un serveur et au moins deux clients lancés en même temps sur la même machine ou le même réseau local.
- Un **code lisible et structuré en paquets** (`modele`, `reseau`, `serveur`, `client`) qui reflète la conception UML.
- Une conception assez propre pour **ajouter un type de besoin sans modifier le code métier existant** (nouvelle sous-classe de `Besoin` ou de `Materiel`, plus sa valeur `type` dans le schéma et la Factory du DAO, section 8.2) et **un type d'événement en ajoutant une valeur à `TypeEvenement`** (et au `CHECK`, section 8.3).

---

3. Contraintes du projet

3.1 Contrainte de temps : 18 heures

Le volume horaire imparti (18 heures cumulées) délimite strictement le périmètre. L'accent est mis sur la solidité de la conception POO, l'héritage, la gestion réseau de base et l'encapsulation. Les fonctionnalités secondaires sont classées P2 ou P3 (section 4) ; les fonctionnalités exclues sont listées en section 5.

3.2 Contraintes techniques

Élément	Choix retenu	Justification
Langage	Java 17	Imposé par le module ; `record`, `switch` modernes disponibles.
Build	Maven (`pom.xml`)	Compilation et dépendances identiques sur toutes les machines du groupe.
Bibliothèques	`sqlite-jdbc`, `jBCrypt`, JavaFX (`org.openjfx`)	Les seules dépendances du projet ; SQLite fait partie des SGBD autorisés ; bcrypt répond à la demande de hachage salé.
Réseau	Sockets TCP (`ServerSocket` / `Socket`) + flux texte	Plus simple qu'une API REST et directement lié au cours. Un thread par client connecté.
Échanges	Une ligne de texte par `Requete` / `Reponse` (`PrintWriter` / `BufferedReader`)	Protocole simple et lisible, testable à la main ; pas de sérialisation Java, donc pas de désérialisation naïve.
Persistance	SQLite via JDBC (`sqlite-jdbc`), `ConnexionBD` + DAO (`serveur.dao`)	Données conservées entre deux lancements, dans un seul fichier, sans serveur à installer.
Interface	JavaFX (vues FXML + contrôleurs)	Interface graphique claire ; la séparation vue / contrôleur reste lisible en POO.
Mots de passe	Hachés avec bcrypt (`jBCrypt`)	Jamais stockés en clair ; bcrypt intègre un sel et un coût réglable contre la force brute.
Images	Logo, bannière et avatar stockés en `byte[]` (BLOB)	Stockés directement dans SQLite ; encodés en Base64 pour passer dans le flux texte.
Dates	`LocalDate` / `LocalDateTime` (`java.time`)	API moderne et immuable, plus sûre que `java.util.Date`.
Gestion de version	Git, un dépôt partagé	Travail en groupe et historique des contributions.

4. Périmètre fonctionnel (développé dans les 18 h)

Priorités : **P1** = MVP, indispensable à la démonstration ; **P2** = prévu, réalisé si le planning le permet ; **P3** = bonus, seulement s'il reste du temps.

4.1 Authentification

ID	Exigence	Priorité
F1	La personne se connecte avec son e-mail et son mot de passe.	P1

ID	Exigence	Priorité
F2	En cas d'échec, un message d'erreur est renvoyé sans préciser si c'est l'e-mail ou le mot de passe qui est faux.	P1
F3	La personne peut se déconnecter.	P1
F4	Les comptes de test sont insérés dans la base SQLite au premier démarrage du serveur ; pas d'inscription en ligne.	P1

4.2 Gestion des événements

ID	Exigence	Priorité
F5	Une personne connectée crée un événement d'un type donné (anniversaire, mariage, soirée, autre) avec nom, description, date, lieu et prix de participation. Elle en devient organisatrice.	P1
F6	La personne voit la liste des événements qu'elle organise et de ceux auxquels elle est invitée.	P1
F7	Un organisateur modifie les informations de l'événement.	P2
F8	Un organisateur supprime l'événement (invitations, besoins et messages associés sont supprimés).	P2
F9	À la création d'un mariage, un budget traiteur (`Consommable`) est ajouté une fois à ses besoins ; un organisateur peut le supprimer s'il n'en veut pas.	P3
F10	Un organisateur ajoute un co-organisateur parmi les personnes existantes (`ajouterOrganisateur()`, sinon `AccesRefuseException`).	P2
F11	Un organisateur ajoute un logo et une bannière à l'événement ; chaque personne peut avoir un avatar (fourni dans les données de test).	P3

4.3 Invitations

ID	Exigence	Priorité
F12	Un organisateur invite une personne existante (par son e-mail). L'invitation est créée avec le statut `EN_ATTENTE` (`inviter()`).	P1
F13	Seul un organisateur de l'événement peut y inviter quelqu'un (`estOrganisateur()`, sinon `AccesRefuseException`).	P1
F14	Une même personne ne peut pas être invitée deux fois au même événement.	P1
F15	L'invité consulte ses invitations et les accepte ou les refuse (`accepter()`, `refuser()`).	P1
F16	Les organisateurs voient l'état de chaque invitation (en attente, acceptée, refusée).	P1

4.4 Besoins et budget (héritage / polymorphisme)

ID	Exigence	Priorité
F17	Un organisateur ajoute ou supprime un besoin (`ajouterBesoin()`, `supprimerBesoin()` ; nom, prix unitaire, quantité) : **consommable** (périssable ou non) ou **matériel** (à rendre ou non, état : neuf, bon, usé, hors service), décliné en **matériel son** (décibels, puissance) ou **matériel lumière** (lumens, couleur).	P1
F18	Un organisateur attribue l'achat d'un besoin à un participant (`acheter()`).	P1

ID	Exigence	Priorité
F19	Un invité ayant **accepté** l'invitation (`estAcceptee()`) peut prendre en charge lui-même un besoin sans acheteur.	P1
F20	L'invité peut changer d'avis tant que l'événement n'est pas passé ; s'il refuse après avoir accepté, ses besoins n'ont plus d'acheteur (règle 3).	P2
F21	Un besoin sans acheteur est acheté par les organisateurs. Les participants voient qui achète quoi (`getNomComplet()` de l'acheteur).	P1
F22	L'application calcule le coût de chaque besoin (`calculerCout()` = prix × quantité) et le budget total de l'événement (`calculerBudget()`).	P1

4.5 Messages

ID	Exigence	Priorité
F23	Un participant envoie un message texte à un autre participant du même événement (`envoyerMessage()`).	P2
F24	Chaque personne consulte les messages qu'elle a envoyés et reçus, triés par date (`afficher()`, `getContenu()`).	P2

4.6 Communication réseau

ID	Exigence	Priorité
F25	Le serveur écoute sur un port configurable et accepte plusieurs clients simultanés (un thread `GestionnaireClient` par connexion).	P1
F26	Chaque action du client est une `Requete` (type + paramètres) ; le serveur renvoie une `Reponse` (succès, message, données).	P1
F27	Les accès concurrents aux données partagées sont protégés (`synchronized` sur le service).	P1
F28	Les données à jour sont obtenues par rafraîchissement manuel (bouton « Actualiser » de l'interface).	P1

4.7 Persistance

ID	Exigence	Priorité
F29	Les données sont enregistrées dans une base SQLite via JDBC (tables `personne`, `evenement`, `organisateur`, `invitation`, `besoin`, `message`).	P1
F30	Chaque modification est écrite immédiatement dans la base ; au redémarrage du serveur, toutes les données sont retrouvées.	P1
F31	Les mots de passe sont stockés sous forme de hachés bcrypt (`mdpHash`) ; la connexion les vérifie avec `BCrypt.checkpw` (`seConnecter()`).	P1

4.8 Règles métier récapitulatives

1. Seuls les organisateurs modifient, suppriment, invitent, ajoutent un co-organisateur et gèrent les besoins de l'événement.
2. Un invité n'agit que sur **ses** invitations.

3. Seuls les participants (organisateurs + invités ayant accepté) peuvent se voir attribuer un besoin ou échanger des messages dans l'événement.
4. Un événement a toujours au moins un organisateur.
5. Toute violation d'une règle lève une exception (`AccesRefuseException` pour les droits, `IllegalStateException` pour les autres règles, ex. besoin pris sans avoir accepté), transformée par le serveur en `Reponse` d'échec avec un message clair.

---

5. Hors périmètre (justifié par les 18 h)

Fonctionnalité	Raison / alternative
Discussions de groupe, pièces jointes	Seuls les messages texte d'une personne à une autre sont prévus.
Temps réel (WebSockets, notifications push)	Remplacé par un rafraîchissement au clic, éventuellement un requêtage périodique simple.
Paiement en ligne	Le prix et le budget sont indicatifs ; aucun paiement n'est géré.
Inscription en ligne, mot de passe oublié	Comptes de test insérés dans la base au premier démarrage.
Chiffrement des échanges (TLS)	Démonstration sur réseau local uniquement.

---

6. Exigences non fonctionnelles

- **Robustesse :** une entrée invalide (date mal formée, e-mail inconnu, prix négatif) affiche un message d'erreur et ne fait jamais planter le client ni le serveur.
- **Réactivité :** côté client, les appels réseau s'exécutent hors du thread JavaFX (`Task`) pour que l'interface ne se fige jamais.
- **Sécurité :** chaque requête est vérifiée côté serveur ; le client n'est jamais considéré comme fiable. Toutes les requêtes SQL utilisent `PreparedStatement` (pas d'injection SQL).
- **Maintenabilité :** attributs `private`, accès par méthodes ; ajouter un type de besoin = ajouter une sous-classe (voir 2.4).
- **Portabilité :** fonctionne sur Windows, macOS et Linux avec un JDK 17 ; Maven récupère la version de JavaFX adaptée au système.
- **Lisibilité :** noms de classes et méthodes en français, cohérents avec les diagrammes UML.

---

7. Architecture

L'application est découpée en quatre paquets :

Paquet	Rôle	Classes principales
`modele`	Objets métier et leurs règles	`Personne`, `Evenement`, `TypeEvenement`, `Invitation`, `StatutInvitation`, `Message`, `Besoin` (+ `Consommable`, `Materiel` → `MaterielSon`, `MaterielLumiere`), `EtatMateriel`, `AccesRefuseException`
`reseau`	Protocole partagé entre client et serveur	`Requete`, `Reponse`, `TypeRequete`
`serveur`	Écoute réseau, contrôle d'accès, persistance	`Serveur`, `GestionnaireClient`, `ServiceEvenements`, `Dao` (interface), `EvenementDao`, `PersonneDao`, `InvitationDao`, `BesoinDao`, `MessageDao`, `ConnexionBD`, `AuthentificationException`
`client`	Connexion au serveur et interface JavaFX	`Client`, `ApplicationClient`, `ControleurConnexion`, `ControleurEvenements`, `ControleurEvenement` (+ vues `.fxml`)

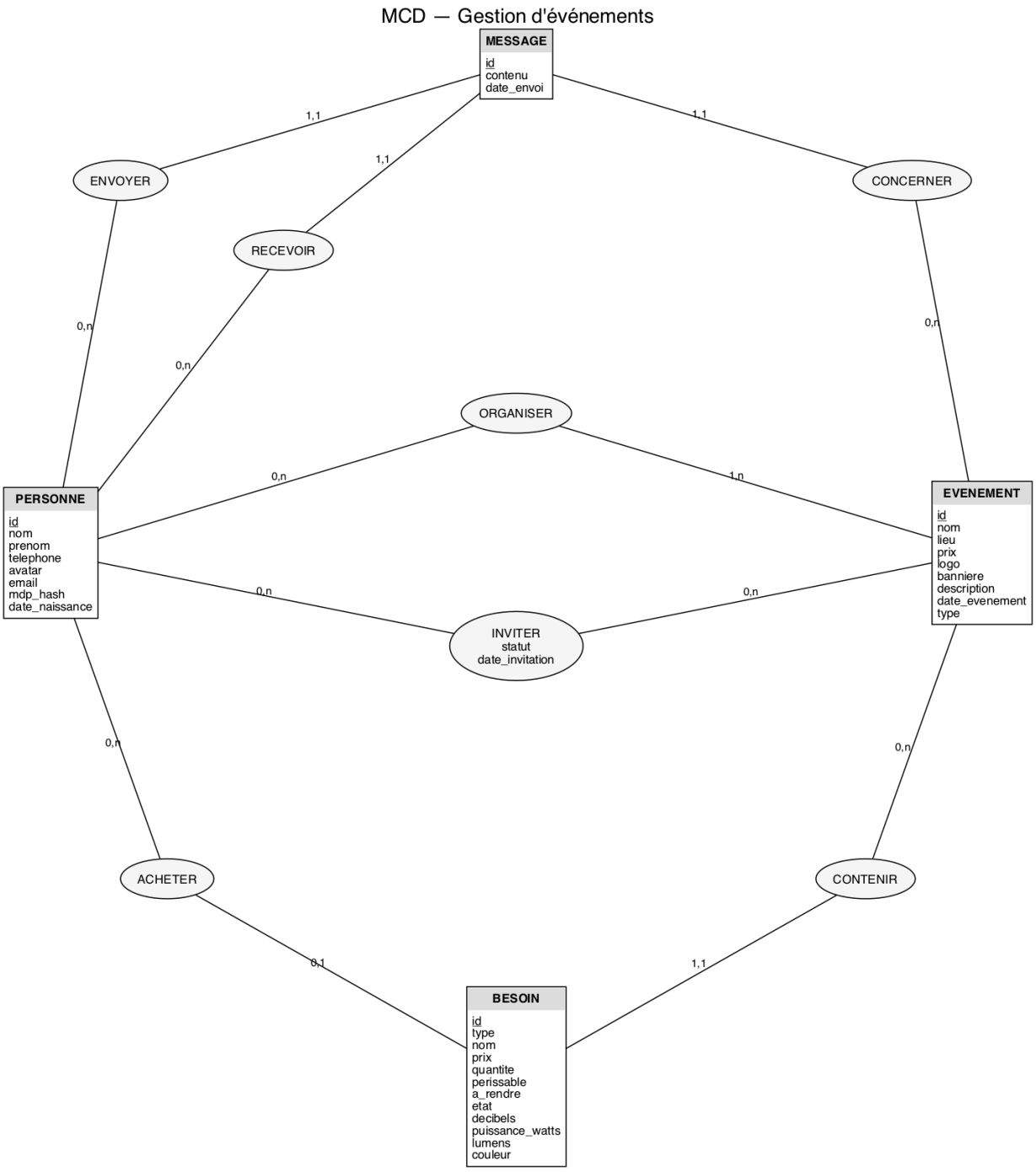
Notions POO mises en œuvre :

- **Héritage :** `Besoin` → `Consommable`, `Materiel` ; `Materiel` → `MaterielSon`, `MaterielLumiere`.
- **Classes abstraites et polymorphisme :** `Besoin` et `Materiel` sont abstraites ; un `Evenement` manipule tous ses besoins via une `List<Besoin>` (`calculerCout()`, `acheter()`), quel que soit leur type.
- **Interface :** `Dao<T>`, implémentée par `EvenementDao` et `PersonneDao` (une classe DAO par table : tout le SQL reste dans les DAO, le service n'en contient aucun).
- **Composition :** un `Evenement` possède ses `Invitation`, ses `Besoin` et ses `Message` (supprimés avec lui).
- **Agrégation :** un `Evenement` référence ses organisateurs (`Personne`), qui existent indépendamment de lui.
- **Énumérations :** `TypeEvenement`, `StatutInvitation`, `EtatMateriel`, `TypeRequete`.
- **Exceptions personnalisées :** `AccesRefuseException` (levée par le modèle, ex. `inviter()`), `AuthentificationException` (levée par le service à la connexion).
- **Threads :** un `GestionnaireClient` (`Runnable`) par client connecté côté serveur ; une `Task` JavaFX par appel réseau côté client.

---

8. Base de données

SGBD : **SQLite** (autorisé par le module, aucune installation). Le schéma est la traduction directe du diagramme de classes (section 9.2). Les livrables de la base (MCD, scripts SQL, optimisation) sont dans le dossier `bdd/`.



8.1 Modèle logique (MLD)

Clés primaires notées `PK`, clés étrangères notées `#`.

Table	Colonnes	Origine dans l'UML
`personne`	`id` PK, `nom`, `prenom`, `telephone`, `avatar` (BLOB), `email` UNIQUE NOT NULL, `mdp_hash` NOT NULL, `date_naissance`	Classe `Personne`
`evenement`	`id` PK, `nom` NOT NULL, `lieu`, `prix`, `logo` (BLOB), `banniere` (BLOB), `description`, `date_evenement` NOT NULL, `type`	Classe `Evenement` + enum `TypeEvenement`

Table	Colonnes	Origine dans l'UML
`organisateur`	`#id_evenement`, `#id_personne` — PK composée	Lien N–N `organisateurs` → table de jonction
`invitation`	`id` PK, `#id_evenement`, `#id_personne`, `statut`, `date_invitation` — UNIQUE (`id_evenement`, `id_personne`)	Classe `Invitation` (lien 1–N `invitations`)
`besoin`	`id` PK, `#id_evenement`, `#id_acheteur` (NULL possible), `type`, `nom`, `prix`, `quantite`, `perissable`, `a_rendre`, `etat`, `decibels`, `puissance_watts`, `lumens`, `couleur`	Hiérarchie `Besoin` (table unique)
`message`	`id` PK, `#id_evenement`, `#id_envoyeur`, `#id_receveur`, `contenu` NOT NULL, `date_envoi` NOT NULL (attribut `date`)	Classe `Message`

Les compositions de l'UML (`Invitation`, `Besoin`, `Message` dans `Evenement`) se traduisent par `ON DELETE CASCADE` sur `id_evenement` (F8). Les clés étrangères sont activées à chaque connexion (`PRAGMA foreign_keys = ON`, désactivé par défaut dans SQLite).

8.2 Héritage : table unique

Toute la hiérarchie `Besoin` (`Consommable`, `MaterielSon`, `MaterielLumiere`) est stockée dans la table `besoin`, avec une colonne `type` (`CONSOMMABLE`, `MATERIEL_SON`, `MATERIEL_LUMIERE`).

- **Pourquoi :** l'écran principal (F19, F21) liste tous les besoins d'un événement, quel que soit leur type : une seule requête, sans jointure.
- **Contrepartie :** quelques colonnes `NULL` (`lumens` n'a de sens que pour du matériel lumière). Des contraintes `CHECK` par type empêchent les combinaisons incohérentes.
- **Côté Java :** le DAO utilise une Factory qui instancie la bonne sous-classe selon la colonne `type`.

8.3 Contraintes

Contrainte	Règle protégée
`UNIQUE (email)` sur `personne`	Un compte par e-mail (F1)
`UNIQUE (id_evenement, id_personne)` sur `invitation`	Pas de double invitation (F14)
`CHECK (statut IN ('EN_ATTENTE', 'ACCEPTEE', 'REFUSEE'))`	Enum `StatutInvitation`
`CHECK (type IN ('ANNIVERSAIRE', 'MARIAGE', 'SOIREE', 'AUTRE'))`	Enum `TypeEvenement`
`CHECK (type IN ('CONSOMMABLE', 'MATERIEL_SON', 'MATERIEL_LUMIERE'))` sur `besoin`	Hiérarchie `Besoin` (8.2)
`CHECK (etat IN ('NEUF', 'BON', 'USE', 'HORS_SERVICE'))` sur `besoin`	Enum `EtatMateriel`
`CHECK (prix >= 0)`, `CHECK (quantite > 0)`	Valeurs métier valides

8.4 Données non stockées

Rien de ce qui se calcule n'est enregistré : le coût d'un besoin (`prix × quantite`), le budget d'un événement, l'âge d'une personne, la liste des participants et le fait qu'un besoin soit pris en charge par un

participant (`acheteur` renseigné) sont calculés à la demande (`calculerCout()`, `calculerBudget()`, `getAge()`, `getParticipants()`, `estAchete()`).

8.5 Index

Au moins deux index, chacun justifié par un `EXPLAIN QUERY PLAN` avant / après (dans `bdd/optimisation.md`) :

Index	Requête accélérée
`idx_invitation_personne ON invitation (id_personne)`	« Mes invitations » (F15)
`idx_besoin_evenement ON besoin (id_evenement)`	Liste des besoins d'un événement (F19, F21, F22)
`idx_message_receveur ON message (id_receveur, date_envoi)`	Messages reçus, triés par date (F24)

L'index créé par `UNIQUE (id_evenement, id_personne)` couvre déjà la liste des invitations d'un événement (F16).

8.6 Transactions

Comme dans le kit de la séance 2, le DAO fait le SQL et le service (`ServiceEvenements`) ouvre et valide la transaction.

- **Création d'un événement (règle 4) :** l'`INSERT` dans `evenement` et l'`INSERT` du créateur dans `organisateur` (plus le budget traiteur d'un mariage, F9) sont validés ensemble, ou pas du tout (`commit` / `rollback`) : un événement n'existe jamais sans organisateur.
- **Refus après acceptation (F20) :** le passage à `REFUSEE` et la libération des besoins de l'invité (`id_acheteur = NULL`) sont validés ensemble, ou pas du tout (`commit` / `rollback`).

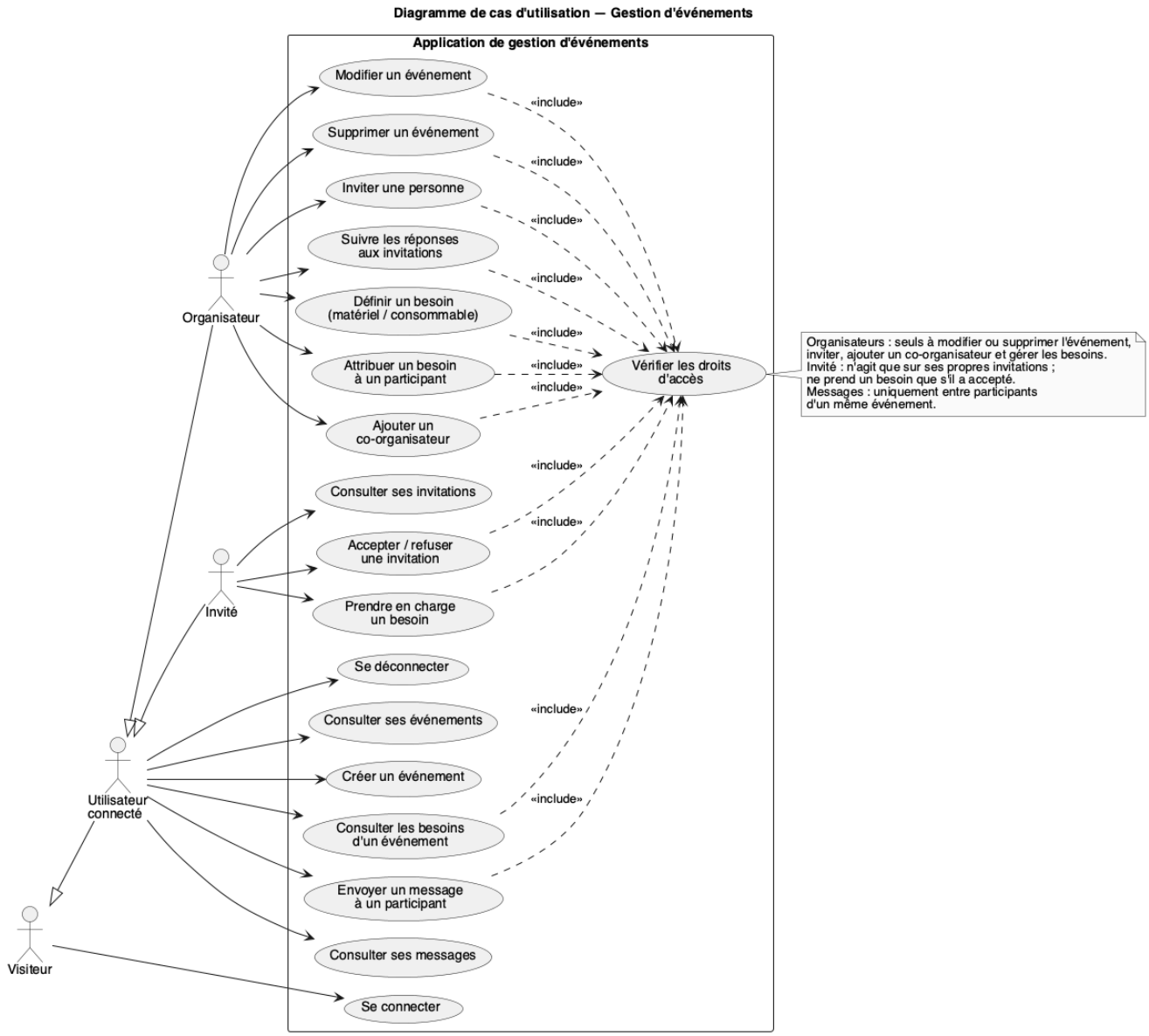
8.7 Accès JDBC

- `ConnexionBD` lit les paramètres dans `db.properties`, qui n'est pas versionné (modèle : `bdd/db.properties.example`).
- Les DAO (`serveur.dao`) concentrent tout le SQL ; toutes les requêtes passent par `PreparedStatement` (jamais de SQL concaténé) et `try-with-resources`.
- `bdd/schema.sql` et `bdd/donnees_test.sql` recréent la base en une commande ; `MainTestBDD` vérifie que l'accès aux données fonctionne.

---

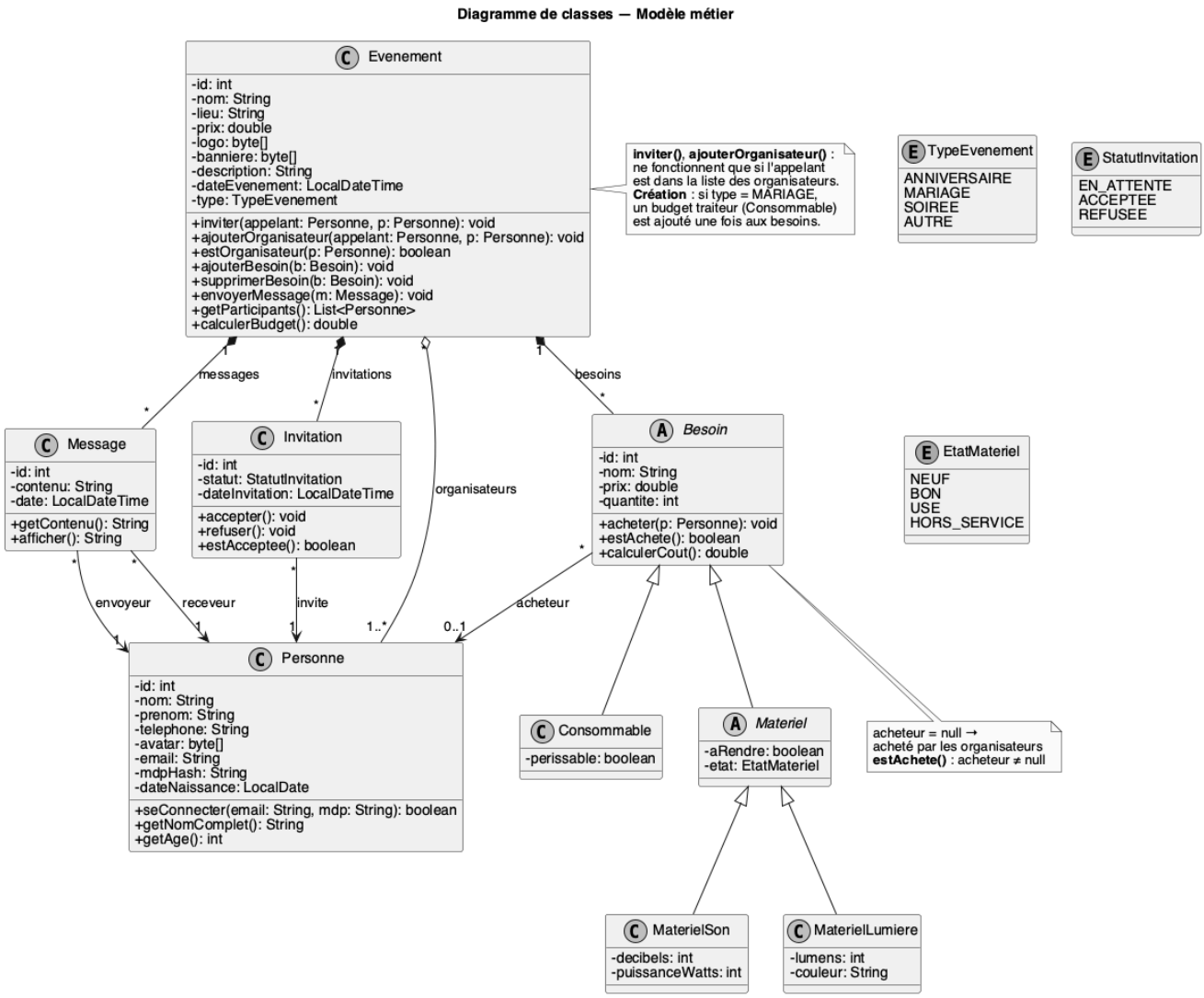
9. Conception UML

9.1 Diagramme de cas d'utilisation



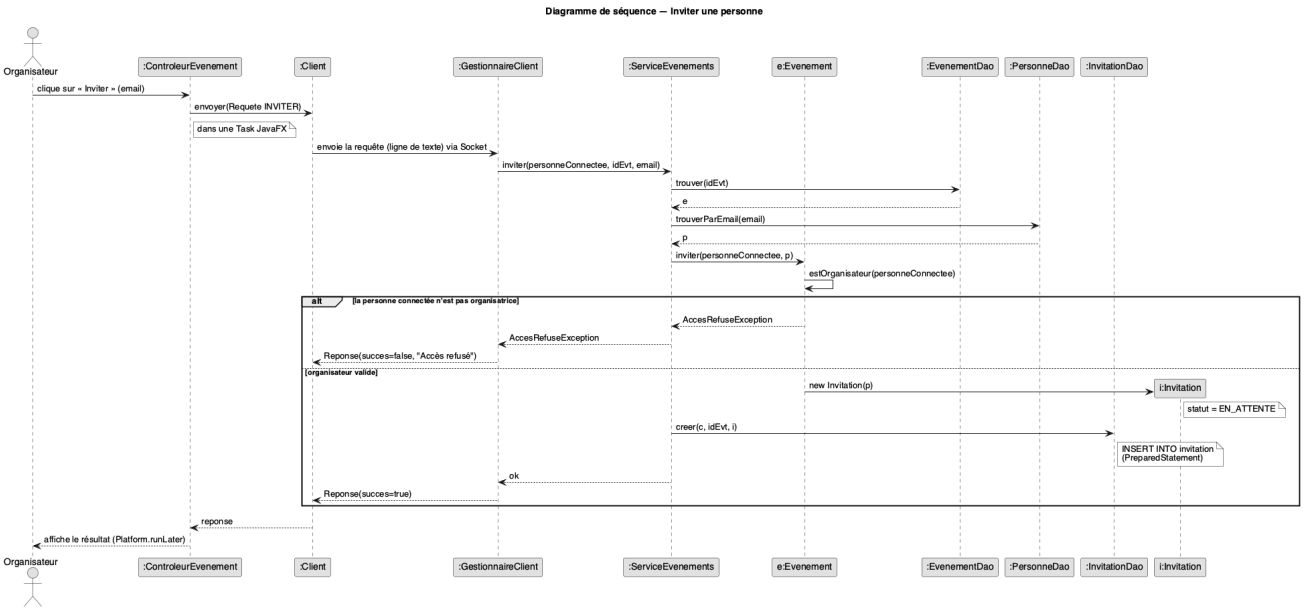
Les actions sensibles incluent toutes le cas « Vérifier les droits d'accès », réalisé côté serveur : par `ServiceEvenements`, ou directement par le modèle quand la règle y est portée (`Evenement.inviter()` vérifie que l'appelant est organisateur, voir 9.3).

9.2 Diagramme de classes



Un `Evenement` est composé de ses invitations, de ses besoins et de ses messages, et agrège ses organisateurs. `Besoin` est abstraite et se spécialise en `Consommable` et `Materiel` (`MaterielSon`, `MaterielLumiere`).

9.3 Diagramme de séquence — Inviter une personne



Ce diagramme montre le trajet complet d'une requête : de l'interface JavaFX jusqu'au serveur, la vérification des droits, l'écriture dans SQLite, puis la réponse renvoyée au client.

---

10. Planning prévisionnel (18 h)

Séance	Date	Durée	Contenu	Livrable
1	06/10	3 h	Groupes, choix du sujet, cahier des charges, UML	**Étape 1** (ce document, complété au jalon 2)
2	20/10	3 h	Base de données : MCD / MLD, `schema.sql`, index + `EXPLAIN`, transaction, `ConnexionBD`, DAO (`serveur.dao`)	**Jalon 2** : dossier `bdd/`, tag `jalon-2`
3	03/11	3 h	Paquet `modele` et logique métier sur la base (`ServiceEvenements`, règles, exceptions)	Règles métier testées avec un `main`
4	17/11	3 h	`reseau` + `serveur` : sockets, threads, protocole texte, bcrypt ; IHM : prototype des écrans et charte de couleurs	Serveur qui authentifie et répond, maquette de l'interface
5	01/12	3 h	Paquet `client` : écrans JavaFX, requêtes P1 puis P2, tests, optimisation, README	Démonstration à deux clients
6	15/12	3 h	Présentation des livrables : démonstration, présentation, débat	**Rendu final**

---

11. Critères de validation

Le projet est considéré comme réussi si le scénario suivant fonctionne avec un serveur et deux clients :

1. Alice se connecte et crée une soirée.
2. Alice invite Bob ; Bob voit l'invitation et l'accepte.

3. Alice ajoute deux besoins : « 6 bouteilles de jus » (consommable, 2 € l'unité) et « 1 enceinte » (matériel son, 30 €).
4. Bob prend en charge l'enceinte ; Alice voit, après actualisation, que Bob l'achète et que le budget total est de 42 €.
5. Bob tente d'inviter quelqu'un à la soirée d'Alice : le serveur refuse avec « Accès refusé ».
6. Le serveur est redémarré : la soirée, l'invitation et les besoins sont toujours là (lus depuis la base SQLite).